BUILD SPEC · V0.1

Context Paging

Tool outputs become handles, not history.
A missing-middle memory layer for tool-using LLM agents —
between active context and the curated brain wiki.

AUTHOR	Todd Ghidaleson · Director, Solution Engineering · Salesforce
AUDIENCE	An implementation agent (Claude Code, or equivalent) tasked with building the MVP
STATUS	Design spec — not yet implemented. Org registry returned no strong matches.
DROP FOLDER	~/AgenticProjects/context-paging/
DATE	2026-06-04
PRIOR ART	MemGPT / Letta (memory blocks); Claude Code auto-memory (curated facts); Aider repo map; Cursor @file

§ Contents

1. Executive summary

Today every tool output (file read, grep result, bash output, API response) accumulates in the agent's context window *forever* — until automatic summary-compaction destroys it lossily and irrecoverably. After ~30 tool calls, 90% of context is "cold" — the agent already moved past it but is still paying full token cost.

Context Paging is a thin harness layer that intercepts every tool result. Outputs above a token threshold are written byte-exact to a content-addressed disk store, and replaced in the agent's context with a structured *stub* (~30–80 tokens) that captures signatures, dependencies, shape, and a one-line summary. A new tool, `recall(stash_id, query?)`, lets the agent page content back when needed — either in full or as a query-extracted slice via a small extractor model.

The system is **lossless** (original bytes preserved on disk), **durable** (content-addressed by SHA-256, so cross-session continuity is automatic via dedup), and **composable** (no model changes, no protocol changes, no rewrite of the agent loop — works on top of Claude Code, Letta, or any tool-using harness). It is the missing layer between MemGPT-style block memory (chat/persona) and Claude Code's auto-memory (curated conclusions): an *evidence layer* with *query-extracted recall*, applied at the tool boundary.

> **HEADLINE NUMBERS**
> A 3,400-token `Read` output collapses to ~80 tokens once stashed. Across a typical 100-tool-call session, expect 70–85% reduction in late-session context size with no loss of recoverability. The first measurable wins appear around the 30th tool call.

2. The problem we're solving

2.1 The accumulation problem

Modern code agents (Claude Code, Cursor agent mode, Aider, Devin, OpenHands) all share the same shape: a long-lived conversation in which every tool call's full output becomes permanent context. Token cost grows linearly with tool calls, but the *useful* portion of that context grows sub-linearly — most outputs are consulted once and never re-read.

2.2 Why current solutions are wrong

MECHANISM	WHAT IT DOES	WHY IT'S NOT ENOUGH
Larger context windows	Push the wall further out	Quadratic attention cost; eventually still hit the wall; doesn't address late-session attention degradation
Summary compaction	LLM summarizes old turns when full	**Lossy and destructive** — original bytes gone forever; agent can't re-derive specifics
Auto-memory (Claude Code)	Persist *conclusions* across sessions	Solves cross-session learning, not in-session evidence accumulation
MemGPT / Letta blocks	OS-style paging of memory blocks	Architected for chat/persona/user; not specifically for tool outputs; whole-block recall, no query-extraction
Subagents	Isolate work in a sub-context	Subagent itself pays full token cost; result still bloats orchestrator

2.3 The gap

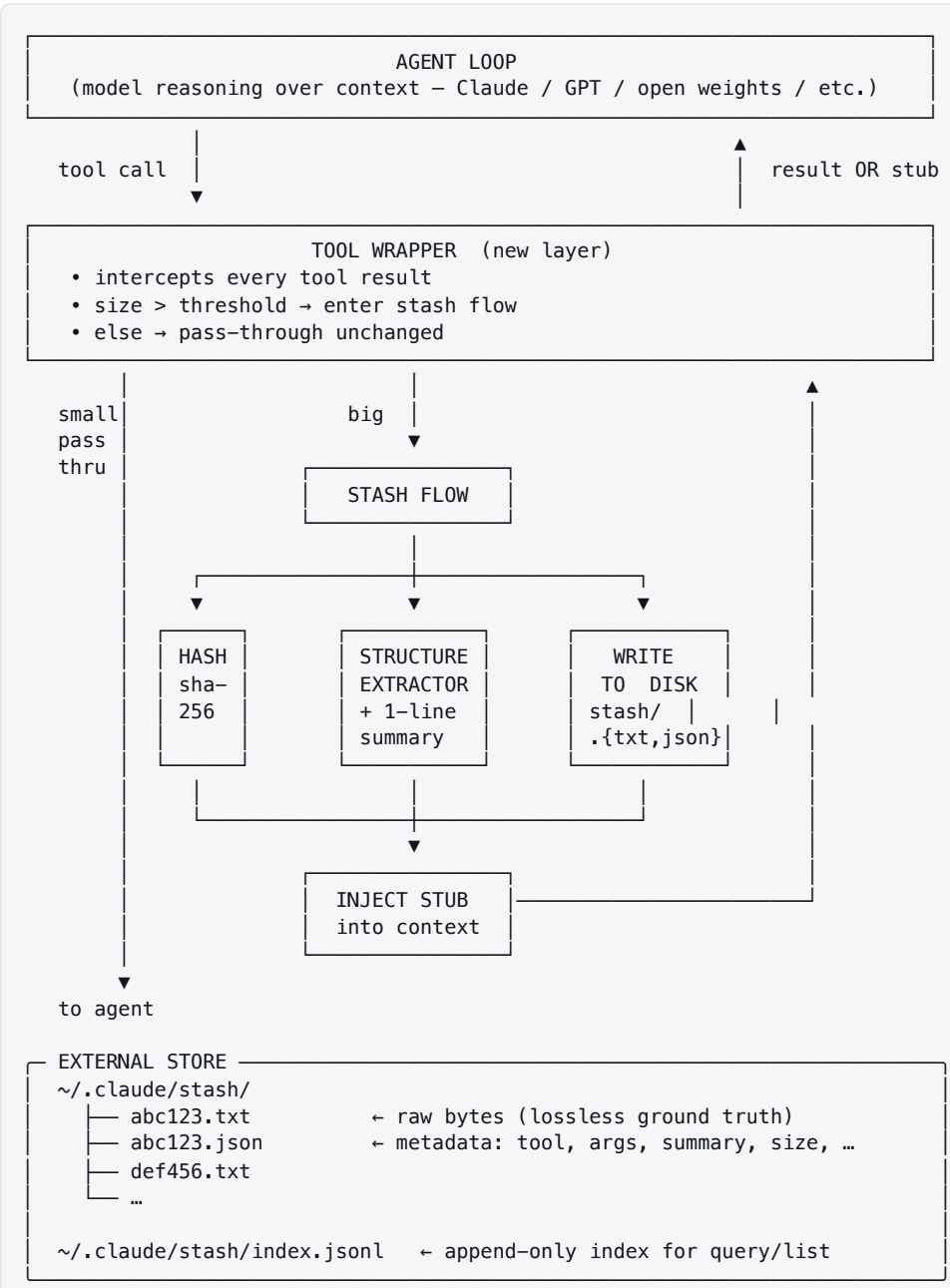
None of the above auto-stash *tool outputs* at the harness boundary, with content-addressed durability and query-extracted recall. That is the gap this spec fills.

3. Core concept in one paragraph

> *Treat the agent's context like RAM and the disk like swap. When a tool returns more than N tokens, write the full output to a content-addressed disk store and replace it in context with a small structured stub: tool name, args, size, signatures/exports, and a one-line summary. Expose a* `recall(stash_id, query?)` *tool that returns either the full content or — if a query is provided — a small slice extracted by a fast model. Stashes persist across sessions; the same content read twice deduplicates by hash. The agent itself decides what's hot vs. cold; the harness just provides the paging primitive.*

4. Architecture & the four-layer model

4.1 Component diagram



4.2 The four-layer memory model

LAYER	LIVES IN	COST	PURPOSE	WHO MANAGES
1. Active context	Conversation tokens	Full	What the agent is reasoning over *right now*	The agent loop
2. Stash stubs	Conversation tokens	~30–80 tok each	Handles to evidence the agent consulted but isn't actively editing	Tool wrapper (auto)
3. Stash store	`~/.claude/stash/`	$0 (disk)	Lossless content-addressed evidence, durable across sessions	Tool wrapper writes; `recall` reads
4. Auto-memory	`~/.claude/projects/<repo>/memory/`	~25KB index loaded at session start	Curated *conclusions* — already exists in Claude Code	The agent (model writes notes)

Layer 2 is the new layer. Layer 3 is the storage backing it. Layers 1 and 4 already exist. The win is the missing middle: *cheap pointers to verbatim evidence*, available across sessions.

5. Data flow — three paths

5.1 Path A — small tool result (pass-through)

```
agent → Read("config.json")
      ← 180 tokens
wrapper: 180 < threshold (500) → return as-is
agent receives: full content, no stash created.
```

Simple cases stay simple. The wrapper is a no-op when output is small.

5.2 Path B — large tool result (auto-stash)

```
agent → Read("auth.go")
      ← 3,400 tokens

wrapper:
  1. token-count(output)  → 3,400 > 500
  2. sha256(output)       → "abc123…"
  3. lookup index.jsonl   → not seen before
  4. structure-extractor(output, tool="Read", args={file: auth.go})
       → { exports: ["VerifyToken/2", "RotateRefresh/1", "ParseHeader/1"],
           imports: ["jwt-go", "redis", "crypto/rand"],
           shape:   "142 lines · 4 funcs · 1 struct (Claims)",
           summary: "JWT verify + refresh-token rotation, Redis-backed blocklist" }
  5. write stash/abc123.txt   (raw bytes, exact)
     write stash/abc123.json  (metadata + structure)
     append index.jsonl       (one line)
  6. inject stub into context (replaces tool result):
       [stash:abc123] Read(auth.go) — 3,400 tok
         exports: VerifyToken(string)→(Claims,error)
                  RotateRefresh(token)→(string,error)
                  ParseHeader(req)→(string,error)
         imports: jwt-go, redis, crypto/rand
         shape:   142 lines · 4 funcs · 1 struct (Claims)
         summary: JWT verify + refresh-token rotation, Redis-backed blocklist
       (call recall("abc123") for full content,
        or recall("abc123", "your question") for a slice)

agent receives: ~80-token stub. Continues reasoning.
```

5.3 Path C — recall

```
agent → recall("abc123")
      ← full 3,400 tokens (from stash/abc123.txt, byte-exact)

agent → recall("abc123", "what does VerifyToken return on expired tokens?")
wrapper:
  1. read stash/abc123.txt
  2. extractor-model(content, query) — small/fast model, ≤200-token output budget
  3. return the extracted slice
      ← "VerifyToken returns (nil, ErrTokenExpired) when the exp claim
         is in the past. Lines 47–58 of auth.go. The blocklist check
         happens before expiry validation, so revoked-but-unexpired
         tokens return ErrRevoked instead."
      ~50 tokens.
```

5.4 Dedup is automatic

Reading the same file twice — in the same session, in another session next week, in a subagent, anywhere — produces the same SHA-256 and reuses the existing stash. No GC required for correctness; pruning is a separate disk-management concern.

6. Stash record schema

6.1 Two files per stash

Every stash is two sibling files in `~/.claude/stash/`:

FILE	FORMAT	PURPOSE
`<hash>.txt`	UTF-8 bytes (or `.bin` for binary)	Verbatim ground truth — never mutated, never rewritten
`<hash>.json`	JSON	Metadata, structure, summary, sighting log

6.2 The metadata JSON

```
{
  "id":          "abc123",                       // first 6 hex of sha256
  "sha256":      "abc123de…fullhex",              // full digest
  "created_at": "2026-06-04T14:22:31Z",
  "size_bytes": 14820,
  "size_tokens": 3400,                           // approx, via tiktoken or char/4
  "tool":        "Read",
  "args":        {"file_path": "/repo/src/auth.go"},
  "structure": {
    "language": "go",
    "exports":  ["VerifyToken(string)→(Claims,error)",
                 "RotateRefresh(token)→(string,error)",
                 "ParseHeader(req)→(string,error)"],
    "imports":  ["jwt-go", "redis", "crypto/rand"],
    "shape":    "142 lines · 4 funcs · 1 struct (Claims)"
  },
  "summary":    "JWT verify + refresh-token rotation, Redis-backed blocklist",
  "stub_depth": "outline",                       // outline | full-toc | minimal
  "sightings":  [
    {"session": "sess_xyz", "ts": "2026-06-04T14:22:31Z"},
    {"session": "sess_abc", "ts": "2026-06-12T09:01:14Z"}
  ],
  "tags":       []                               // optional, agent-assignable
}
```

6.3 The append-only index

`~/.claude/stash/index.jsonl` — one JSON object per line, append-only. Written under file lock. Used for:

- Cross-session lookup (`recall` by partial id, by tool, by file path)
- `list_stashes(filter)` tool implementations
- Pruning / GC tooling (LRU, size cap)

Index entries are denormalized snapshots of the JSON metadata at write time. Treat `<hash>.json` as authoritative if there is a conflict.

7. Stub format — structured, not prose

7.1 Why structured matters

A prose stub like `"JWT-related stuff in auth.go"` is useless for routing — the agent cannot tell whether to recall. A structured stub with signatures, dependencies, and shape gives the agent enough to make the recall/skip decision *without* reading content. The stub is a routing primitive, not a content replacement.

7.2 Three stub depths

DEPTH	TOKENS	USE WHEN
`minimal`	~30	One-shot reads, throwaway grep results, transient API responses
`outline` *(default)*	~80	Code/config files, structured documents — the common case
`full-toc`	~250	Files the agent expects to revisit; opt-in via `stash(content, depth="full-toc")`

7.3 Stub templates by tool type

READ (CODE)

```
[stash:<id>] Read(<path>) — <tok>
  exports: <sig list>
  imports: <module list>
  shape:   <lines> · <funcs> · <classes>
  summary: <1-line>
```

READ (MARKDOWN / TEXT)

```
[stash:<id>] Read(<path>) — <tok>
  outline: <# headings>, <# sections>
  toc:     <top 5 headings>
  summary: <1-line>
```

GREP / GLOB

```
[stash:<id>] Grep("<pat>") — <n> matches
  files:   <top-5 paths> …
  summary: <1-line>
```

BASH / API CALL

```
[stash:<id>] Bash("<cmd>") — exit <n>, <tok>
  stdout-head: <first 80 chars>
  stderr-head: <first 80 chars> (if non-empty)
  summary:     <1-line>
```

7.4 Hard rule: the stub never fabricates

> **INVARIANT**
>
> The structure-extractor only emits facts that are **parseable from the raw output** (signatures from the AST, headings from the markdown tree, file paths from the grep result). The 1-line `summary` is the only freeform field, and it is generated under a strict prompt that forbids speculation. The agent is told: *treat the summary as routing-grade, not authoritative; recall before acting.*

8. Recall semantics

8.1 The `recall` tool

```
recall(
  stash_id: string,            // required — full hash or unique prefix
  query?:   string,             // optional — extract only the slice that answers this
  mode?:    "full" | "extract" | "lines",  // default: "extract" if query, else "full"
  range?:   {start: int, end: int},        // optional line range, only with mode="lines"
)
  → string                       // the recalled content (full, slice, or line range)
```

8.2 Three recall modes

MODE	RETURNS	COST	USE CASE
`full`	Verbatim file contents	Full token cost (one-time)	Editing the file; the agent *needs* the source
`extract`	Query-extracted slice (50–200 tok)	One small-model call + tiny output	Answering a specific question against the content
`lines`	Specific line range	Trivial	"Show me lines 47–58"

8.3 The extractor prompt

Run on a fast model (Haiku-class). Strict template:

```
SYSTEM: You are a content extractor. Given source content and a query,
return ONLY the portion of the content that directly answers the query.
Quote verbatim where possible. If the answer is not in the content,
respond exactly: "NOT_IN_CONTENT". Do not speculate. Maximum 200 tokens.

USER:
<source content here, full bytes>

QUERY: <the agent's question>
```

The `NOT_IN_CONTENT` sentinel is critical: the agent should treat that as "the stub may have misled me — re-strategize" rather than "no answer exists."

8.4 Recall is biased on by the system prompt

> HARNESS INSTRUCTION (ADDED TO SYSTEM PROMPT)
>
> *"When you have a stash stub in context, treat the stub as routing-grade information for deciding which stash to consult — never as authoritative content. If your decision depends on specifics not in the stub, call `recall`. The cost of an unnecessary recall is one tool call; the cost of guessing wrong is a real bug."*

9. Composition with existing systems

9.1 With Claude Code's auto-memory

Auto-memory entries can *cite* stashes via wikilink: `[[stash:abc123]]`. A conclusion stays cheap (one fact in `MEMORY.md`), and the evidence chain is recoverable on demand. This gives Claude something today's auto-memory cannot offer: **verifiable provenance** for its remembered learnings.

9.2 With subagents

A subagent's work product becomes a list of stash IDs plus a small report. Today, a subagent that reads 10 files and reports back forces the orchestrator to absorb a 2,000-token summary of those reads. With stashing, the subagent returns:

```
{ "summary": "auth flow uses JWT + Redis blocklist; refresh handled in middleware",
  "evidence": ["stash:abc123", "stash:def456", "stash:7a9c12"] }
```

The orchestrator pays handle-cost (~30 tok each) and can drill in via `recall` only on the stashes it cares about. Subagents stop being context-budget-busters.

9.3 With MCP

Stashes are URI-addressable: `stash://abc123`. Any MCP-aware tool can reference them. Future direction: a `stash` MCP server that exposes `list`, `get`, `extract`, and `prune` as resources, allowing other agents and tooling to share the same stash store.

9.4 With Letta / MemGPT memory blocks

Stash records are lower-level than memory blocks. A memory block can pin a stash ("this is the canonical auth.go evidence") and recall through the same path. The systems are complementary: Letta's blocks structure *working memory*; the stash store backs it with *durable evidence*.

10. Implementation plan (MVP → v1)

10.1 MVP — zero new infra, harness-local

Target: working prototype in <500 LOC, zero external services, runs entirely on the local filesystem.

1. **Tool wrapper.** Hook into the harness's `PostToolUse` path (Claude Code exposes this). For each tool result, decide stash vs pass-through.
2. **Stash directory.** Create `~/.claude/stash/` on first use. Write `<hash>.txt` + `<hash>.json` + append `index.jsonl`.
3. **Structure extractor.** Per-language: tree-sitter for code, markdown-it for md, regex for grep/bash. Falls back to "first/last 100 chars + line count" when no parser matches.
4. **Summarizer prompt.** One Haiku-class call per stash, ≤50-token output. Template: *"In one sentence: what is in this content? Be specific. No speculation."*
5. **The `recall` tool.** Registered with the agent's tool list. Reads from disk; runs the extractor model when `query` is provided.
6. **System prompt addendum.** The "treat stubs as routing-grade" instruction.

10.2 v1 — quality & ergonomics

- **Three stub depths** (`minimal` / `outline` / `full-toc`). Agent can request a richer stub at stash time.
- **Stash search.** `list_stashes(filter)` tool: filter by tool, by file path, by date, by tag.
- **Cross-session continuity.** Surface relevant stashes from prior sessions at session start ("you stashed 12 things from this repo last week — here are the top 5 by recent access").
- **Tag & pin.** Agent can `pin(stash_id)` to mark a stash as long-term-relevant; pinned stashes survive GC.
- **LRU pruning.** Background sweep: drop unpinned stashes older than N days when total store exceeds size cap.

10.3 v2 — share & scale

- **MCP server.** Stash store exposed as an MCP server so multiple agents share state.
- **Team stash.** Optional remote stash backend (S3 / SharePoint / git) so a team's evidence is shared. Privacy-aware: stashes carry an explicit `visibility` field.
- **Embedding index.** For very large stash stores, build a vector index over summaries to enable semantic `list_stashes` .

10.4 Suggested file layout for the implementation

```
~/AgenticProjects/context-paging/
├── README.md
├── SPEC.pdf                    ← this document
├── src/
│   ├── wrapper.py              ← tool-result interceptor
│   ├── store.py                ← disk store: write/read/index
│   ├── extractor/
│   │   ├── code.py             ← tree-sitter-based per-language
│   │   ├── markdown.py
│   │   └── fallback.py
│   ├── summarizer.py           ← Haiku-class summarization
│   ├── recall.py               ← the recall tool implementation
│   └── prompts/
│       ├── summarize.txt
│       └── extract.txt
├── tests/
│   ├── test_wrapper.py
│   ├── test_dedup.py
│   ├── test_extractor.py
│   ├── test_recall_full.py
│   └── test_recall_extract.py
└── bench/
    └── token_savings.py        ← measures baseline vs paged context
```

11. Edge cases & gotchas

11.1 Mutation

If a file is edited, the next `Read` hashes differently → new stash created. **This is correct behavior.** The old stash remains as historical evidence. Don't try to "update" a stash; the content-addressed model relies on immutability.

11.2 Binary / non-UTF-8 content

Detect via byte sniff. Store as `<hash>.bin` with metadata flagging `"binary": true`. Stub omits the `summary` field and shows `"binary content, <n> bytes"`. `recall` returns a base64-encoded blob plus a warning. Most agents won't actually want this back; the stash exists for replay/debugging.

11.3 Secrets & sensitive content

> **SECURITY**
>
> The stash store is on disk in plaintext. Anything the agent reads can be replayed later. **Three controls:**
>
> 1. **Path denylist.** Files matching `.env`, `id_*`, `credentials.*`, `*.pem`, etc. are stashed in a separate `secret-stash/` tree with mode 0700, or skipped entirely (configurable).
> 2. **Content scanner.** Run a fast secret-detector (e.g., truffleHog patterns) at stash time. If a secret is detected, redact in the stub *and* in the stored content; flag with `"secret_redacted": true`.
> 3. **No cross-org sync by default.** Stash store is machine-local until explicitly opted into team sharing.

11.4 Hash collisions

SHA-256, 256 bits — collision is not a practical concern. Index is keyed by full hash; the 6-hex display ID is for the agent's convenience. If two short IDs collide in display, prompt the agent to disambiguate with more hex.

11.5 Stub drift vs. content

The stub captures structure and summary at *stash time*. If the agent recalls weeks later, the stash content is byte-identical to when it was stashed (immutable), but the file on disk in the original location may have changed. The agent must understand: **recall returns the historical snapshot, not the current file.** If the agent wants the current state, it should `Read` the file again — which will produce a new stash if it differs.

11.6 Threshold tuning

The 500-token threshold is a starting heuristic. Below it, stashing adds friction (an extra round-trip for content the agent could have just read inline). Above it, savings dominate. Make it configurable and instrument it; the right value is empirical.

11.7 Summarizer cost

Each large tool result costs one extra small-model call (≈$0.0001 on Haiku). For a 100-tool-call session with ~70 stashes, that's ≈$0.007 — negligible against the savings on the main model.

11.8 The agent ignoring the stub

Some agents may aggressively recall everything, defeating the savings. Mitigations: (a) make the stub *good* (structured, sufficient for routing); (b) the system-prompt instruction to recall only when needed; (c) measure recall-rate per session and surface it as a debugging signal.

12. Verification & success metrics

12.1 Functional tests

- Reading the same file twice produces one stash (dedup).
- `recall(id)` returns byte-exact original content.
- `recall(id, query)` returns a slice ≤ 200 tokens, returning `NOT_IN_CONTENT` when the answer isn't present.
- Stash survives session restart; cross-session lookup works.
- Path denylist correctly skips/secures sensitive files.
- Binary content stored and round-tripped correctly.

12.2 Quantitative success metrics

METRIC	BASELINE (NO PAGING)	TARGET (WITH PAGING)
Late-session context size (after 50 tool calls)	~80–120K tokens	≤30K tokens
Tokens spent on tool-output history	~85% of context	≤25% of context
Successful task completion rate	baseline	≥ baseline (no regression)
Recall-call rate per session	n/a	5–15 (mostly query-extract mode)
Compaction events per long session	1–3	0 (paging defers compaction)

12.3 Qualitative checks

- Run a multi-day investigation (50+ tool calls across 3 sessions). Agent should reference prior-session evidence by recalling stashes, not by re-reading files.
- Adversarial: feed the agent a file with deliberately misleading structure. Confirm the agent recalls before acting on stub-based assumptions.
- Run with auto-memory enabled. Confirm auto-memory entries cite stashes when they reference specific evidence.

13. Open questions / future work

1. **Best stub depth heuristic.** Should the wrapper guess depth from tool/file type, or always use `outline` and let the agent upgrade? MVP picks the latter; v1 should A/B test.
2. **Streaming tools.** Long-running bash commands stream output. Stash on completion vs. checkpoint stashes mid-stream? MVP defers — stash on completion only.
3. **Non-deterministic tools.** Two API calls with the same args may return different bodies. Hashing dedups identical responses; differing responses get distinct stashes (correct).
4. **Image / multimodal outputs.** Out of scope for MVP. v2 should treat images analogously: stash blob + caption stub.
5. **Stash-aware skill discovery.** Skills could declare "this skill expects stash IDs as input" — would let multi-step pipelines hand off evidence by reference.

6. **Compression.** Large stashes can be gzipped on disk. `recall` decompresses transparently. Worth doing once stash store exceeds ~1GB.
7. **Stash garbage collection policy.** LRU + size cap is a starting point. Pinned stashes are immortal. Agents may want to mark stashes as `"transient"` to opt out of long-term retention.
8. **Privacy & multi-tenant safety.** If team-shared, redaction and access control must be enforced before write — not after.

14. Glossary

TERM	DEFINITION
Stash	A persisted, content-addressed copy of a tool output, with metadata.
Stub	The 30–250-token structured handle that replaces the full output in the agent's context.
Recall	The act of pulling content back from the stash store. Three modes: `full`, `extract`, `lines`.
Extract mode	Recall variant that runs a small LLM extractor over the stashed content to return only a query-relevant slice.
Stub depth	How verbose the stub is: `minimal`, `outline`, `full-toc`.
Pin	Mark a stash as long-term-relevant; survives LRU pruning.
Dedup	Same content → same SHA-256 → same stash. Reading a file twice never doubles storage.
Auto-memory	Existing Claude Code feature: model writes durable conclusions to `~/.claude/projects/<repo>/memory/`. Layer 4 in this spec's model.
Memory block	Letta/MemGPT primitive — a discrete, structured unit of agent memory. Higher-level than a stash; can pin/cite stashes.